RESTATE (this subpart)

Implement **WAH compression (word_size=8)** for **one bitmap column string** (e.g., "00010...") by chunking into **payload_size = 7** bits, emitting either:

- **literal word** if mixed bits
- **fill word** if all-0 or all-1 (even if only 1 chunk)

…and handle the **final short chunk** by **right-padding with 0s** before classifying.

Slide anchors: runs/literals + words [S18], WAH fill vs literal layout + worked example [S42–S53].

QUICK REVIEW: your current setup is the right "starting point"

✅ In your compress_index, you already:

- read rows
- compute num_columns
- build columns = [''.join(line[i] for line in lines) ...]

That matches the "compress each bitmap column independently" requirement.

⚠️ One spec trap (note for later): output_path is supposed to be a **directory** and filename uses the naming scheme. Your current code treats output_path as a file path. Don't fix now if you're time-boxing; just flag it.

CORRECT WAH FACTS (word_size = 8) — burn these in

Word size W = 8 → **payload_size = W−1 = 7** [S43]

Each output "WAH word" is **8 characters** (since you're writing bits-as-chars).

Literal word format (8 chars)

- **Leading bit:** 0 means literal
- **Next 7 bits:** the payload bits (exact chunk, padded if final)
- Example: 0 0101101 → "00101101" (8 chars)

Fill word format (8 chars)

- **Leading bit:** 1 means fill/run [S44]
- **Second bit:** fill value (0 for zeros fill, 1 for ones fill) [S44]
- **Remaining 6 bits:** run length **in number of 7-bit chunks** (not raw bits) [S48–S51]
- Example: 1 0 000011 → zeros fill of length 3 chunks

Key correction to your current mental model:

- The run length is **count of payload-chunks**, not "fill the payload itself."
- If your column has 1001 bits, you **do not** encode "fill 1000 bits"; you encode fills for the number of **full 7-bit chunks** that are all same bit.

TIMED BLOCKS (do in order, minimal but "defensible")

Block 1 (5 min) — "Proof you own it": write down the formats

Write on paper (or comments) exactly:

- W=8, payload=7
- literal: 0 + 7 payload
- fill: 1 + fillbit + 6-bit count
- count measured in **chunks of 7**

This is straight from [S43–S51].

Block 2 (8 min) — Column → chunks mental drill (no code)

Take a sample column string length N.

- Split into chunks of 7 from left to right.
- Last chunk: pad right with 0s to length 7.
- For each chunk: classify as all-0, all-1, or mixed.

Do this once by hand so coding is mechanical.

Block 3 (15 min) — Implement

wah_compress(col, word_size=8)

core loop

Goal: a loop that **walks chunks** and builds output words, merging consecutive fills.

Block 4 (10 min) — Add counters + tiny tests

- Track fill_words, literal_words.
- Test with columns that force:
  - all zeros
  - alternating bits (all literals)
  - zeros run then mixed then zeros run

WHERE TO START IN CODE (the core logic scaffold)

You asked for redundant, explicit, line-by-line, heavily-commented logic. Here's a **scaffold** that is "explainable in 60 seconds" and maps to slides.

This is structured code, but still a scaffold: you'll fill in the tiny details (like formatting count bits). That keeps it "owned" and within your PoW expectations.

```
def wah_compress(col_bits: str, word_size: int = 8):
    """
    WAH compress a single bitmap COLUMN string (e.g. '0001010...'),
    using bits-as-chars output (not packed bits).
    Slides: [S43–S51], runs/literals [S18].
    """

    payload = word_size - 1            # [S43]
    max_count_bits = word_size - 2     # for W=8, this is 6 bits (fill header
uses 2 bits)

    out_words = []                     # list of 8-char strings
    fill_words = 0
    literal_words = 0

    # Helper to chunk column into payload-sized pieces, padding final chunk
on the right with '0' [S52–S53]
    # NOTE: padding affects classification (final could become all-0 =>
should be fill).
    chunks = []
    i = 0
    while i < len(col_bits):
        chunk = col_bits[i:i+payload]
        if len(chunk) < payload:
            chunk = chunk + ('0' * (payload - len(chunk)))   # pad right with 0s
(spec + [S52–S53])
        chunks.append(chunk)
        i += payload

    # Now scan chunks left->right, emitting either literals or fills, merging
consecutive fills of same type.
    run_type = None     # None or '0' or '1'
    run_len = 0         # number of consecutive fill chunks of run_type
```

```
    def flush_run():
        # Convert current run_type/run_len into one or more fill words.
        # If run_len is too large for 6 bits, you may need multiple fill words.
        nonlocal run_type, run_len, fill_words

        if run_len == 0:
            return

        # For W=8, count field is 6 bits => max 63.
        # Emit in blocks of at most 63 chunks.
        while run_len > 0:
            take = min(run_len, (2 ** max_count_bits) - 1)

            # Fill word format: 1 + run_type + COUNT(6 bits)  [S44]
            # COUNT is number of payload-chunks covered by this fill word.
            count_bits = format(take, f"0{max_count_bits}b")
            out_words.append('1' + run_type + count_bits)

            fill_words += 1
            run_len -= take

        run_type = None

    for chunk in chunks:
        # classify chunk as fill or literal [S18]
        all0 = (chunk == '0' * payload)
        all1 = (chunk == '1' * payload)

        if all0 or all1:
            bit = '0' if all0 else '1'

            # If we were building a run of the SAME bit, extend it; otherwise
flush previous run and start new.
            if run_type is None:
                run_type = bit
                run_len = 1
            elif run_type == bit:
                run_len += 1
```

```
        else:
            flush_run()
            run_type = bit
            run_len = 1

    else:
        # Literal breaks any run: flush run first, then emit literal [S47]
        flush_run()

        # Literal format: 0 + payload bits  [S43–S44]
        out_words.append('0' + chunk)
        literal_words += 1

# End: flush any pending run
flush_run()

# Return BOTH compressed stream and stats (needed for writeup)
compressed_stream = ''.join(out_words)
return compressed_stream, fill_words, literal_words
```

What you should "care about" while implementing

- **Chunking correctness** (payload=7, right-pad last chunk). If this is wrong, everything is wrong.
- **Run length measured in chunks**, not bits.
- **Flush rule:** a literal forces a run flush; switching fill type forces flush.
- **Single fill chunk still becomes a fill word** (spec requirement).
- **Count overflow:** count field limited (for W=8, max 63). If your dataset is big, you must split runs (easy + defendable).

MICRO-TESTS (fast, evidence you understand it)

Run these on wah_compress:

1. **All zeros (forces a fill)**

- input: "0000000" (exactly one payload chunk)
- expected: one fill word: starts with "10" and count ends with ...001 (count=1)

2. **Mixed bits (forces literal)**

- input: "0001000"
- expected: one literal word starts with "0" and then exact payload "001000"? (whatever the 7-bit input is)

3. **Two chunks, same fill**

- input: "0"*14
- expected: one fill word with count=2 (binary 000010)

4. **Final short chunk padding check**

- input length not multiple of 7, e.g. "111" → becomes "1110000" after padding
- classification: mixed → literal (because contains both 1 and 0)

UNLOCK PROMPT (so I can do “surgical correction” on your real output)

Paste:

- your current wah_compress attempt (even partial)
- a 2–4 line expectation statement: *“Given column X and word_size=8, I should output Y WAH words of length 8 each, concatenated…”*
- 1 self-check: *“I’m unsure whether count is chunks vs bits / whether last padded chunk can become a fill…”*

Then I’ll point out the **first** incorrect step (if any) and give the minimal fix + one test to confirm.